

河北工程大学 2024-2025 年大学计算机 (2) 重点内容

Python程序设计基础

一、Python 简介

1. Python 是什么？

- Python 是一种**高级编程语言**：它的语法接近自然语言，更容易学习和阅读。
- Python 是一种**解释型语言**：代码在执行时逐行翻译成机器码，无需预先编译。
- Python 是一种**面向对象的语言**：支持面向对象的编程范式。
- Python 是**免费开源的**：任何人都可以免费使用和分发 Python。

2. 为什么学习 Python？

- **易学易用**：语法简洁，代码可读性高，适合初学者入门。
- **应用广泛**：可用于 Web 开发、数据科学、人工智能、网络爬虫、自动化脚本等众多领域。

二、Python 基础语法

1. Python 注释

- 注释是代码中用于解释说明的部分，不会被程序执行。
- **单行注释**：以井号 # 开头。

```
# 这是一个单行注释  
name = "Alice" # 这也是一个注释
```

- **多行注释**：可以使用三个单引号 ''' 或三个双引号 """ 包裹。

2. Python 行与缩进

- Python 代码通常**一行写一条语句**。
- **缩进非常重要**！Python 使用缩进来表示代码块的层级关系（例如，循环体、函数体、条件语句块等）。
- 通常使用 **4 个空格**作为一级缩进。同一代码块的缩进必须一致。

三、Python 标识符和关键字

1. 标识符 (Identifiers)

- 标识符是用来给变量、函数、类、模块等命名的。
- 命名规则：
 - 第一个字符必须是字母（大小写均可）或下划线 _。
 - 其余字符可以是字母、数字或下划线。
 - 区分大小写。例如，`myVariable` 和 `myvariable` 是两个不同的标识符。
 - 不能使用 Python 的关键字作为标识符。
- 示例：

```
# 合法标识符
age = 20
user_name = "Bob"
_count = 0
MyClass = "Some class" # 虽然合法，但不推荐用于变量

# 非法标识符
# 2_starts_with_number = 10 # 不能以数字开头
# user-name = "Alice"      # 不能包含连字符
# class = "Math"           # class 是关键字
```

2. 关键字 (Keywords)

- 关键字是 Python 中具有特殊含义的保留字，不能用作标识符。
- 一些常见的关键字: `if`, `else`, `elif`, `while`, `for`, `def`, `class`, `return`, `import`, `True`, `False`, `None`, `and`, `or`, `not`, `try`, `except`, `finally`, `with`, `is`, `in`, `lambda`, `global`, `nonlocal`, `pass`, `break`, `continue`, `yield`, `assert`, `del`, `async`, `await`。

四、Python 变量

1. 变量赋值

- 变量是用来存储数据的内存位置的名称。
- 使用赋值运算符 `=` 来创建变量并给它赋值。
- 语法：变量名 = 值

```
x = 10          # 整数变量
name = "Alice"  # 字符串变量
pi = 3.14        # 浮点数变量
is_student = True # 布尔变量
```

- 变量在使用前必须先被赋值。

2. 动态类型

- Python 是一种动态类型语言。这意味着你不需要预先声明变量的类型。
- 变量的类型是在运行时根据赋给它的值自动确定的。
- 同一个变量名可以先后指向不同类型的数据。

```
my_var = 100          # my_var 是整数类型
print(type(my_var)) # 输出: <class 'int'>

my_var = "Hello"      # my_var 现在是字符串类型
print(type(my_var)) # 输出: <class 'str'>
```

3. 多重赋值

- Python 允许同时为多个变量赋多个值。

```
a, b, c = 1, 2, "hello"
print(a) # 输出: 1
print(b) # 输出: 2
print(c) # 输出: hello
```

- 也可以将多个变量赋为同一个值。

```
x = y = z = 0
print(x) # 输出: 0
print(y) # 输出: 0
print(z) # 输出: 0
```

4. 删除变量

- 可以使用 del 关键字删除一个或多个变量。

```
my_variable = 10
print(my_variable) # 输出: 10

del my_variable
# print(my_variable) # 再次尝试访问会引发 NameError, 因为变量已被删除
```

五、Python 数据类型

Python 中有多种内置的数据类型，主要可以分为数字、序列（字符串、列表、元组）、映射（字典）、集合等。

0.5.1 数字类型 (Numeric Types)

- **整数 (int)**: 表示整数值, 可正可负, 没有大小限制。例如: 10, 0, -25。
- **浮点数 (float)**: 表示带有小数点的数字, 或使用科学计数法表示的数字。例如: 3.14, -0.001, 2.5e2 (表示 $2.5 \times 10^2 = 250.0$)。
- **布尔类型 (bool)**: 只有两个值: **True** (真) 和 **False** (假)。在运算中, **True** 可以被当作 1, **False** 可以被当作 0。布尔类型是整数类型的子类型。

```
x = 10
y = 3.14
is_active = True
is_greater = 5 > 3 # is_greater 的值为 True
```

- **类型转换**: 可以使用内置函数进行类型转换。

– **int(x)**: 将 x 转换为整数。如果 x 是浮点数, 则截断小数部分。如果 x 是布尔值, **True** 转为 1, **False** 转为 0。如果 x 是表示数字的字符串, 则转为对应整数。

– **float(x)**: 将 x 转换为浮点数。

– **str(x)**: 将 x 转换为字符串。

– **bool(x)**: 将 x 转换为布尔值。对于数字, 0、0.0 为 **False**, 其他为 **True**。对于序列 (字符串、列表等), 空为 **False**, 非空为 **True**。**None** 也为 **False**。

```
num_str = "123"
num_int = int(num_str) # num_int 为 123
```

```
val_float = 3.99
val_int = int(val_float) # val_int 为 3 (小数部分被截断)
```

```
print(float(10))      # 输出: 10.0
print(str(100))       # 输出: "100"
print(bool(0))        # 输出: False
print(bool(-5))       # 输出: True
print(bool(""))       # 输出: False
print(bool("Hello"))  # 输出: True
```

0.5.2 字符串 (Strings)

字符串是字符的序列, 用于表示文本信息。Python 中的字符串是**不可变的**, 意味着一旦创建, 就不能修改其内容。

- **创建字符串**: 可以使用单引号 '、双引号 " 或三引号 ''' / """" (用于多行字符串)。

```
s1 = 'Hello, Python!'
s2 = "你好，世界！"
s3 = """这是一个
多行字符串。"""
```

- 访问字符串中的值 (索引和切片):

–索引 (Indexing): 通过方括号 [] 和索引值访问单个字符。索引从 0 开始，最后一个字符的索引是 长度-1。也支持负数索引，-1 表示最后一个字符，-2 表示倒数第二个，以此类推。

```
my_string = "Python"
print(my_string[0]) # 输出: P
print(my_string[2]) # 输出: t
print(my_string[-1]) # 输出: n
```

–切片 (Slicing): 通过 [start:end:step] 获取子字符串。

- * start: 起始索引 (包含)。如果省略，默认为 0。
- * end: 结束索引 (不包含)。如果省略，默认为字符串末尾。
- * step: 步长 (默认为 1)。

```
my_string = "HelloPython"
print(my_string[0:5]) # 输出: Hello (从索引0到4)
print(my_string[:5]) # 输出: Hello (从开头到索引4)
print(my_string[5:]) # 输出: Python (从索引5到末尾)
print(my_string[1:7:2]) # 输出: eLP (从索引1到6, 步长为2)
print(my_string[::-1]) # 输出: nohtyPolleH (反转字符串)
```

- 字符串基本操作:

- 连接 (+): 将两个字符串连接起来。
- 重复 (*): 将字符串重复多次。
- 长度 (len()): 返回字符串中字符的个数。
- 成员检查 (in, not in): 检查一个子字符串是否存在于另一个字符串中。

```
s1 = "Hello"
s2 = "World"
print(s1 + " " + s2) # 输出: Hello World
print(s1 * 3) # 输出: HelloHelloHello
print(len(s1)) # 输出: 5
print("ell" in s1) # 输出: True
print("xyz" not in s1) # 输出: True
```

- 常用字符串方法：字符串有很多内置方法来执行常见任务。

- `upper()` / `lower()`: 转换为大写/小写。
- `strip()` / `lstrip()` / `rstrip()`: 去除两端/左端/右端的空白字符（或指定字符）。
- `split(sep=None)`: 按指定分隔符 `sep` 将字符串分割成列表。如果 `sep` 未指定或为 `None`, 则以空白字符为分隔符。
- `join(iterable)`: 将可迭代对象 (如列表) 中的字符串元素连接成一个新的字符串, 以当前字符串作为分隔符。
- `find(sub[, start[, end]])`: 查找子字符串 `sub` 第一次出现的位置, 如果找不到则返回 -1。
- `replace(old, new[, count])`: 将字符串中的 `old` 子串替换为 `new` 子串, 可选参数 `count` 指定替换次数。
- `startswith(prefix)` / `endswith(suffix)`: 检查字符串是否以指定前缀/后缀开头/结尾。
- `isdigit()` / `isalpha()` / `isalnum()`: 检查字符串是否只包含数字/字母/字母或数字。

```
text = " Hello, Python! "  
print(text.upper())           # 输出: " HELLO, PYTHON! "  
print(text.strip())          # 输出: "Hello, Python!"  
print(text.split(', '))      # 输出: [' Hello', ' Python! ']  
words = ["Python", "is", "fun"]  
print(" ".join(words))       # 输出: "Python is fun"  
print(text.find("Python"))    # 输出: 9 (P的索引)  
print(text.replace("Python", "World")) # 输出: " Hello, World! "  
print("image.jpg".endswith(".jpg")) # 输出: True
```

- 字符串格式化：将变量的值插入到字符串中。

- **f-strings (格式化字符串面值)** (推荐): 在字符串前加上 `f` 或 `F`, 用花括号 `{}` 包裹变量名或表达式。

```
name = "Alice"  
age = 30  
message = f"My name is {name} and I am {age} years old."  
print(message) # 输出: My name is Alice and I am 30 years old.  
print(f"2 + 3 = {2 + 3}") # 输出: 2 + 3 = 5
```

- **`str.format()` 方法:**

```
message = "My name is {} and I am {} years old.".format(name, age)  
print(message)  
message = "My name is {n} and I am {a} years old.".format(n=name, a=age)  
print(message)
```

0.5.3 列表 (Lists)

列表是 Python 中最常用的数据结构之一，它是一个**有序且可变**的元素集合。列表中的元素可以是不同类型的。

- **创建列表**：使用方括号 `[]`，元素之间用逗号 `,` 分隔。

```
empty_list = []
numbers = [1, 2, 3, 4, 5]
mixed_list = [1, "hello", 3.14, True]
nested_list = [[1, 2], ["a", "b"]] # 嵌套列表
```

- **访问列表元素 (索引和切片)**：与字符串类似。

```
my_list = ['p', 'y', 't', 'h', 'o', 'n']
print(my_list[0])    # 输出: 'p'
print(my_list[-1])   # 输出: 'n'
print(my_list[2:5])  # 输出: ['t', 'h', 'o']
print(my_list[:3])   # 输出: ['p', 'y', 't']
print(my_list[3:])   # 输出: ['h', 'o', 'n']
```

- **修改列表 (因为列表是可变的)**:

- 修改单个元素: `my_list[index] = new_value`
- 修改切片: `my_list[start:end] = new_iterable`

```
my_list = [1, 2, 3, 4]
my_list[0] = 100      # 修改第一个元素
print(my_list)        # 输出: [100, 2, 3, 4]
```

```
my_list[1:3] = [20, 30] # 修改索引1和2的元素
print(my_list)          # 输出: [100, 20, 30, 4]
```

```
my_list[1:3] = [8, 9, 10, 11] # 替换切片，长度可以改变
print(my_list)                # 输出: [100, 8, 9, 10, 11, 4]
```

- **列表常用方法**:

- `append(x)`: 在列表末尾添加一个元素 `x`。
- `insert(i, x)`: 在索引 `i` 处插入元素 `x`。
- `extend(iterable)`: 将可迭代对象中的所有元素追加到列表末尾。
- `remove(x)`: 移除列表中第一个值为 `x` 的元素。如果元素不存在会报错。

-pop([i]): 移除并返回索引 i 处的元素（默认为最后一个元素）。
-clear(): 移除列表中的所有元素。
-index(x[, start[, end]]): 返回第一个值为 x 的元素的索引。
-count(x): 返回元素 x 在列表中出现的次数。
-sort(key=None, reverse=False): 对列表进行原地排序（会修改原列表）。
-reverse(): 反转列表中的元素顺序（原地反转）。
-copy(): 返回列表的浅拷贝。

```
fruits = ["apple", "banana"]
fruits.append("cherry")
print(fruits)          # 输出: ['apple', 'banana', 'cherry']

fruits.insert(1, "orange")
print(fruits)          # 输出: ['apple', 'orange', 'banana', 'cherry']

more_fruits = ["grape", "mango"]
fruits.extend(more_fruits)
print(fruits)          # 输出: ['apple', 'orange', 'banana', 'cherry', 'grape', 'mango']

fruits.remove("banana")
print(fruits)          # 输出: ['apple', 'orange', 'cherry', 'grape', 'mango']

popped_fruit = fruits.pop(1) # 移除索引为1的元素 'orange'
print(popped_fruit)        # 输出: orange
print(fruits)              # 输出: ['apple', 'cherry', 'grape', 'mango']

numbers = [3, 1, 4, 1, 5, 9, 2, 6]
numbers.sort()
print(numbers)            # 输出: [1, 1, 2, 3, 4, 5, 6, 9]（原地排序）
numbers.reverse()
print(numbers)            # 输出: [9, 6, 5, 4, 3, 2, 1, 1]（原地反转）

print(len(fruits))        # 输出: 4（获取列表长度）
```

- **列表推导式 (List Comprehensions):** 一种简洁的创建列表的方式。

```
# 创建一个0到4的平方数的列表
squares = []
for x in range(5):
    squares.append(x**2)
print(squares) # 输出: [0, 1, 4, 9, 16]
```



```
# 使用列表推导式
squares_comprehension = [x**2 for x in range(5)]
print(squares_comprehension) # 输出: [0, 1, 4, 9, 16]

# 带条件的列表推导式
even_numbers = [x for x in range(10) if x % 2 == 0]
print(even_numbers) # 输出: [0, 2, 4, 6, 8]
```

0.5.4 元组 (Tuples)

元组与列表类似，也是一个有序的元素序列。但元组是不可变的 (immutable)，一旦创建，其内容就不能被修改。

- **创建元组：**使用圆括号 ()，元素之间用逗号，分隔。如果只有一个元素，需要在元素后面加上逗号。

```
empty_tuple = ()
my_tuple = (1, "hello", 3.14)
single_element_tuple = (42,) # 注意逗号！
another_tuple = 1, 2, 3 # 括号可以省略 (元组打包)
```

- **访问元组元素 (索引和切片)：**与列表和字符串完全相同。

```
my_tuple = ('p', 'y', 't', 'h', 'o', 'n')
print(my_tuple[0])    # 输出: 'p'
print(my_tuple[-1])   # 输出: 'n'
print(my_tuple[1:4])  # 输出: ('y', 't', 'h')
```

- **不可变性：**元组的元素不能被修改、添加或删除。

```
my_tuple = (1, 2, 3)
# my_tuple[0] = 100 # 这会引发 TypeError: 'tuple' object does not support item assignment
# my_tuple.append(4) # 元组没有 append 方法
```

- **元组的用途：**

- 当数据集合不应被修改时。
- 作为字典的键（因为键必须是不可变的）。
- 函数返回多个值时，实际上是返回一个元组。

- **元组常用操作/方法：**

- 连接 (+) 和 重复 (*): 与列表类似, 会创建新的元组。
- count(x): 返回元素 x 在元组中出现的次数。
- index(x): 返回第一个值为 x 的元素的索引。
- len(): 获取元组长度。

```
t1 = (1, 2, 3)
t2 = (4, 5)
print(t1 + t2) # 输出: (1, 2, 3, 4, 5)
print(t1 * 2)  # 输出: (1, 2, 3, 1, 2, 3)
```

```
my_tuple = (1, 2, 2, 3, 4, 2)
print(my_tuple.count(2)) # 输出: 3
print(my_tuple.index(3)) # 输出: 3
print(len(my_tuple))    # 输出: 6
```

- 元组打包与解包 (Packing and Unpacking):

```
# 打包: 将多个值赋给一个元组变量
packed_tuple = 1, "Alice", 25
print(packed_tuple) # 输出: (1, 'Alice', 25)

# 解包: 将元组的元素分别赋给多个变量
id_val, name, age = packed_tuple
print(id_val) # 输出: 1
print(name)   # 输出: Alice
print(age)    # 输出: 25
```

0.5.5 字典 (Dictionaries)

字典是 Python 中的一种**无序**（在 Python 3.7+ 版本中, 字典会记住插入顺序, 但通常我们不依赖这个顺序进行编程）的键值对 (key:value) 集合。每个键都是唯一的, 并且必须是**不可变类型**（如字符串、数字或元组）。值可以是任意类型。字典是**可变的**。

- 创建字典: 使用花括号 {}, 键值对之间用逗号 , 分隔, 键和值之间用冒号 : 分隔。

```
empty_dict = {}
student = {"name": "Bob", "age": 20, "major": "Computer Science"}
person = {
    "name": "Alice",
    "age": 30,
    "city": "New York",
    "grades": [85, 90, 78]
```

```
}
```

- 访问字典中的值：通过键来访问对应的值。

```
student = {"name": "Bob", "age": 20}
print(student["name"]) # 输出: Bob
print(student["age"]) # 输出: 20

# 如果键不存在, 直接用 [] 访问会报错 KeyError
# print(student["gender"]) # 会引发 KeyError

# 使用 get() 方法访问, 如果键不存在, 可以返回 None 或指定的默认值
print(student.get("name")) # 输出: Bob
print(student.get("gender")) # 输出: None (因为 gender 键不存在)
print(student.get("gender", "Unknown")) # 输出: Unknown
```

- 添加或修改字典条目:

```
student = {"name": "Bob"}
student["age"] = 21 # 添加新键值对
print(student) # 输出: {'name': 'Bob', 'age': 21}

student["name"] = "Robert" # 修改已有键的值
print(student) # 输出: {'name': 'Robert', 'age': 21}
```

- 删除字典条目:

- del dictionary[key]: 删除指定键的条目。
- dictionary.pop(key[, default]): 删除并返回指定键的值。如果键不存在且未提供默认值, 会报错。
- dictionary.popitem(): 删除并返回字典中最后插入的一个键值对 (在 Python 3.7+)。在早期版本中, 是随机删除一个。
- dictionary.clear(): 清空字典中的所有条目。

```
student = {"name": "Alice", "age": 22, "major": "Math"}
del student["major"]
print(student) # 输出: {'name': 'Alice', 'age': 22}

age = student.pop("age")
print(age) # 输出: 22
print(student) # 输出: {'name': 'Alice'}
```

```

student["city"] = "London"
student["country"] = "UK"
item = student.popitem() # 删除并返回最后插入的 ('country', 'UK')
print(item)              # 输出: ('country', 'UK')
print(student)           # 输出: {'name': 'Alice', 'city': 'London'}

```

• 字典常用方法:

- keys(): 返回一个包含字典所有键的视图对象。
- values(): 返回一个包含字典所有值的视图对象。
- items(): 返回一个包含字典所有 (键, 值) 元组的视图对象。
- update(other_dict): 用另一个字典的键值对更新当前字典。
- copy(): 返回字典的浅拷贝。
- len(): 获取字典中键值对的数量。

```

student = {"name": "Carol", "age": 25, "major": "Physics"}
print(list(student.keys())) # 输出 (顺序可能不同, 但在3.7+通常是插入顺序): ['name', 'age', 'major']
print(list(student.values())) # 输出: ['Carol', 25, 'Physics']
print(list(student.items())) # 输出: [('name', 'Carol'), ('age', 25), ('major', 'Physics')]

print(len(student))         # 输出: 3

```

• 遍历字典:

```

student = {"name": "David", "age": 28}

# 遍历键
for key in student.keys(): # 或者直接 for key in student:
    print(key)

# 遍历值
for value in student.values():
    print(value)

# 遍历键值对
for key, value in student.items():
    print(f"{key}: {value}")

```

六、Python 输入和输出

1. 输入 (Input)

- 使用内置函数 `input()` 从用户那里获取输入。
- `input()` 函数会将用户输入的所有内容都作为字符串返回。
- 可以在 `input()` 函数中提供一个提示字符串。

```
name = input("请输入你的名字: ")
print(f"你好, {name}!")
```

```
age_str = input("请输入你的年龄: ")
print(f"你输入的年龄是: {age_str}, 类型是: {type(age_str)}")
```

- 将输入转换为数字: 如果需要数字类型的输入, 需要使用 `int()` 或 `float()` 进行转换。

```
age_str = input("请输入你的年龄: ")
age_int = int(age_str) # 将字符串转换为整数
print(f"明年你就 {age_int + 1} 岁了。")
```

```
price_str = input("请输入商品价格: ")
price_float = float(price_str) # 将字符串转换为浮点数
print(f"商品价格是: {price_float:.2f}") # 保留两位小数输出
```

- 如果转换失败 (例如, 用户输入了非数字字符), 程序会抛出 `ValueError` 异常。

2. 输出 (Output)

- 使用内置函数 `print()` 向控制台输出信息。
- `print()` 可以输出一个或多个对象, 对象之间默认用空格分隔。

```
print("Hello, World!")
x = 10
y = 20
print("x =", x, "y =", y) # 输出: x = 10 y = 20
```

- `sep` 参数: 可以指定多个输出对象之间的分隔符。

```
print(1, 2, 3, 4, sep='---') # 输出: 1---2---3---4
print("a", "b", "c", sep=', ') # 输出: a, b, c
```

- **end 参数**: 可以指定 `print()` 函数输出完毕后的结束符, 默认为换行符 `\n`。

```
print("第一行", end=' ')\nprint("也在第一行") # 输出: 第一行 也在第一行 (在同一行)\n\nprint("这是第一行。")\nprint("这是第二行。") # 默认情况下, 每条 print 语句输出后会换行
```

- **格式化输出**:

– **f-strings** (推荐, 已在字符串部分介绍):

```
name = "Alice"\nage = 30\nprint(f"姓名: {name}, 年龄: {age}")
```

– **str.format()** 方法 (已在字符串部分介绍):

```
print("姓名: {}, 年龄: {}".format(name, age))
```

七、运算符与表达式详解

表达式是由操作数（如变量、常量）和运算符组成的序列, 运算后会产生一个值。

1. 算术运算符用于执行基本的数学运算。

- **+**: 加法运算。例如: `3 + 5` 结果为 8。对于字符串、列表、元组, 是连接操作。
- **-**: 减法运算。例如: `10 - 4` 结果为 6。
- *****: 乘法运算。例如: `6 * 7` 结果为 42。对于字符串、列表、元组, 是重复操作。
- **/**: 除法运算, 结果总是浮点数。例如: `9 / 2` 结果为 4.5。
- **//**: 整除（地板除）, 结果是商的整数部分（向下取整）。例如: `9 // 2` 结果为 4; `-9 // 2` 结果为 -5。
- **%**: 取模（取余）, 返回除法的余数。例如: `9 % 2` 结果为 1; `9 % 4` 结果为 1。
- ******: 幂运算。例如: `2 ** 3` 结果为 8 (即 2^3)。

```
a = 10\nb = 3\nprint(a + b) # 13\nprint(a - b) # 7\nprint(a * b) # 30\nprint(a / b) # 3.333...\nprint(a // b) # 3
```

```
print(a % b)    # 1
print(a ** b)   # 1000
```

2. 赋值运算符用于给变量赋值。

- **=**: 基本赋值运算符，将右操作数的值赋给左操作数。例如: `x = 5`。
- **复合赋值运算符**: 将算术运算或位运算与赋值结合起来。

+=: 加法赋值, `x += 2` 相当于 `x = x + 2`。

-=: 减法赋值, `x -= 2` 相当于 `x = x - 2`。

***=**: 乘法赋值, `x *= 2` 相当于 `x = x * 2`。

/=: 除法赋值, `x /= 2` 相当于 `x = x / 2`。

//=: 整除赋值, `x //= 2` 相当于 `x = x // 2`。

%=: 取模赋值, `x %= 2` 相当于 `x = x % 2`。

****=**: 幂赋值, `x **= 2` 相当于 `x = x ** 2`。

&=, **|=**, **^=**, **>>=**, **<<=**: 位运算赋值 (见下文位运算符)。

```
c = 10
c += 5  # c 现在是 15
print(c)
c *= 2  # c 现在是 30
print(c)
```

3. 比较运算符 (关系运算符) 用于比较两个值，返回布尔值 `True` 或 `False`。

- **==**: 等于。如果两个操作数的值相等，则为真。例如: `5 == 5` (`True`), `5 == 6` (`False`)。
- **!=**: 不等于。如果两个操作数的值不相等，则为真。例如: `5 != 6` (`True`)。
- **>**: 大于。
- **<**: 小于。
- **>=**: 大于或等于。
- **<=**: 小于或等于。

```
x = 5
y = 10
print(x == y)  # False
print(x != y)  # True
print(x < y)   # True
print(x >= 5)  # True
```

4. 逻辑运算符用于连接或修改布尔表达式。

- **and**: 逻辑与。如果两个操作数都为真，则结果为真。否则为假。

– 运算规则: 如果第一个表达式为 **False**, 则整个表达式的结果就是第一个表达式的值, 不再计算第二个表达式 (短路求值)。否则, 结果是第二个表达式的值。

- **or**: 逻辑或。如果两个操作数中至少有一个为真, 则结果为真。否则为假。

– 运算规则: 如果第一个表达式为 **True**, 则整个表达式的结果就是第一个表达式的值, 不再计算第二个表达式 (短路求值)。否则, 结果是第二个表达式的值。

- **not**: 逻辑非。如果操作数为真, 则结果为假; 如果操作数为假, 则结果为真。

```
a = True
b = False
print(a and b)  # False
print(a or b)   # True
print(not a)    # False

# 短路求值示例
print(0 and (1/0)) # 输出: 0 (1/0 不会执行, 避免了 ZeroDivisionError)
print(5 or (1/0))  # 输出: 5 (1/0 不会执行)
```

5. 成员运算符用于测试序列 (如字符串、列表、元组) 中是否包含某个成员。

- **in**: 如果成员存在于序列中, 则为真。
- **not in**: 如果成员不存在于序列中, 则为真。

```
my_list = [1, 2, 3, 'a']
print(2 in my_list)      # True
print('b' in my_list)    # False
print('a' not in my_list) # False

my_string = "hello"
print('h' in my_string)  # True
```

6. 身份运算符用于比较两个对象的内存地址是否相同, 即它们是否是同一个对象。

- **is**: 如果两个变量引用同一个对象, 则为真。
- **is not**: 如果两个变量引用不同的对象, 则为真。

```
x = [1, 2, 3]
y = [1, 2, 3]
```



```

z = x

print(x is y)      # False (x 和 y 是内容相同但不同的列表对象)
print(x == y)      # True (x 和 y 的内容相同)
print(x is z)      # True (x 和 z 指向同一个列表对象)
print(x is not y)  # True

# 对于小的整数和一些字符串, Python为了效率可能会缓存它们, 导致它们 is 结果为 True
a = 10
b = 10
print(a is b)      # 通常为 True (取决于Python实现和数值大小)

```

注意: `is` 比较的是对象的身份, 而 `==` 比较的是对象的值。通常我们关心的是值是否相等, 所以多用 `==`。

7. 位运算符 (了解即可, 初学者考试中可能不常考) 位运算符把数字看作是二进制数来进行计算。

- `&` (按位与): 如果两个相应的二进制位都为 1, 则结果位为 1, 否则为 0。
- `|` (按位或): 如果两个相应的二进制位中至少有一个为 1, 则结果位为 1, 否则为 0。
- `^` (按位异或): 如果两个相应的二进制位不同, 则结果位为 1, 否则为 0。
- `~` (按位取反): 对数据的每个二进制位取反, 即 0 变 1, 1 变 0。($\bar{x}$ 结果是 $-(x+1)$)
- `<<` (左移): 将数字的二进制位向左移动指定的位数, 右边空出的位用 0 填充。相当于乘以 2 的幂。
- `>>` (右移): 将数字的二进制位向右移动指定的位数, 左边空出的位对于正数用 0 填充, 对于负数 (通常) 用符号位填充。相当于整除以 2 的幂。

```

m = 60 # 60 = 0011 1100 (二进制)
n = 13 # 13 = 0000 1101 (二进制)

print(m & n) # 12 (0000 1100)
print(m | n) # 61 (0011 1101)
print(m ^ n) # 49 (0011 0001)
print(~m)    # -61
print(m << 2) # 240 (1111 0000) (60 * 2^2)
print(m >> 2) # 15 (0000 1111) (60 // 2^2)

```

8. 运算符优先级 (从高到低, 部分常用) 当一个表达式中包含多个运算符时, Python 会根据运算符的优先级来决定计算顺序。可以使用圆括号 `()` 来改变运算顺序。

1. `()`: 括号, 具有最高优先级。

2. ******: 幂运算。

3. *****, **/**, **//**, **%**: 乘、除、整除、取模。

4. **+**, **-**: 加、减 (二元运算符)。

5. **<<**, **>>**: 左移、右移。

6. **|**: 按位或。

7. **==**, **!=**, **>**, **>=**, **<**, **<=**, **is**, **is not**, **in**, **not in**: 比较运算符、身份运算符、成员运算符。

8. **not**: 逻辑非。

9. **and**: 逻辑与。

10. **or**: 逻辑或。

11. **=**, **+=**, **-=**, ***=**, **/=**, **//=**, **%=**, ****=**, **&=**, **|=**, **^=**, **>>=**, **<<=**: 赋值运算符 (优先级最低)。

建议: 如果不确定优先级, 或者想让代码更清晰, 请使用圆括号 **()**。

```
result = 3 + 4 * 2 # 乘法优先, result = 3 + 8 = 11
print(result)
```

```
result = (3 + 4) * 2 # 括号优先, result = 7 * 2 = 14
print(result)
```

八、流程控制语句详解

流程控制语句用于控制程序代码的执行顺序。

1. **条件语句 if/elif/else** 条件判断语句用于根据一个或多个条件是否成立来执行不同的代码块。

- **基本结构:**

if 条件表达式1:

当条件表达式1为**True**时执行的代码块

elif 条件表达式2: # 可选, 可以有0个或多个**elif**

当条件表达式1为**False**且条件表达式2为**True**时执行的代码块

else: # 可选, 最多一个**else**

当以上所有条件表达式都为**False**时执行的代码块

- **示例:**

```

score = 85
if score >= 90:
    print("优秀")
elif score >= 80: # 因为 score = 85, 这个条件满足
    print("良好")
elif score >= 60:
    print("及格")
else:
    print("不及格")
# 输出: 良好

```

- 注意事项:

- 每个条件后面必须有冒号 `:`。
- 冒号后面的代码块必须**缩进** (通常是 4 个空格), 表示这些代码属于该条件。
- `elif` 和 `else` 子句是可选的。
- 条件表达式可以是任何结果为布尔值 (`True` 或 `False`) 的表达式。在 Python 中, 非零数字、非空字符串/列表等被视为 `True`, 而 `0`、`None`、空字符串/列表等被视为 `False`。

- 嵌套 if 语句: if 语句内部可以包含另一个 if 语句。

```

num = 10
if num > 0:
    print("数字是正数")
    if num % 2 == 0:
        print("并且是偶数")
    else:
        print("并且是奇数")
elif num == 0:
    print("数字是零")
else:
    print("数字是负数")

```

2. while 循环语句当给定条件为 `True` 时, 重复执行一个代码块。

- 基本结构:

```

while 条件表达式:
    当条件表达式为True时执行的循环体代码块
    (通常在循环体内需要有改变条件表达式真假性的语句, 以避免死循环)
else: # 可选
    当循环条件变为False时执行的代码块 (如果循环被break中断, 则不执行)

```

- 示例:

```
# 打印0到4的数字
i = 0
while i < 5:
    print(i)
    i += 1 # 更新循环变量, 否则会无限循环
print("循环结束")
# 输出:
# 0
# 1
# 2
# 3
# 4
# 循环结束

# while-else 示例
count = 0
while count < 3:
    print(f"Count is {count}")
    count += 1
else:
    print("Loop finished normally.") # 循环正常结束时执行

count = 0
while count < 3:
    print(f"Count is {count}")
    if count == 1:
        break # 中断循环
    count += 1
else:
    print("Loop finished normally.") # 此处 else 不会执行, 因为循环被 break
```

- 死循环: 如果 `while` 的条件表达式永远为 `True`, 循环将无限执行下去, 称为死循环。需要避免。可以使用 `Ctrl+C` 来中断一个死循环的程序。

3. `for` 循环语句用于遍历序列 (如列表、元组、字符串) 中的每个元素, 或者其他可迭代对象。

- 基本结构:

```
for 迭代变量 in 可迭代对象:
    对每个元素执行的代码块
else: # 可选
```

当循环遍历完所有元素后执行的代码块（如果循环被**break**中断，则不执行）

- 遍历序列:

```
# 遍历列表
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
# 输出:
# apple
# banana
# cherry

# 遍历字符串
my_string = "Python"
for char in my_string:
    print(char, end=" ") # 输出: P y t h o n

# 遍历元组
my_tuple = (10, 20, 30)
for item in my_tuple:
    print(item)
```

- **range()** 函数: 常用于生成一个整数序列, 配合 **for** 循环使用。

- **range(stop)**: 生成从 0 到 **stop-1** 的整数序列。
- **range(start, stop)**: 生成从 **start** 到 **stop-1** 的整数序列。
- **range(start, stop, step)**: 生成从 **start** 到 **stop-1** 的整数序列, 步长为 **step**。

```
# 打印0到4的数字
for i in range(5): # 相当于 range(0, 5)
    print(i)
```

```
# 打印2到5的数字
for i in range(2, 6):
    print(i)
```

```
# 打印0到10之间步长为2的数字
for i in range(0, 11, 2):
    print(i) # 输出: 0, 2, 4, 6, 8, 10
```

- 遍历字典:

```
my_dict = {"name": "Alice", "age": 30}
# 遍历键 (默认)
for key in my_dict:
    print(key, my_dict[key])

# 遍历值
for value in my_dict.values():
    print(value)

# 遍历键值对
for key, value in my_dict.items():
    print(f"{key} -> {value}")
```

- for-else 示例:

```
numbers = [1, 3, 5]
for num in numbers:
    print(f"Processing {num}")
else:
    print("All numbers processed.") # 循环正常结束

numbers = [1, 2, 3, 4]
for num in numbers:
    if num == 2:
        print("Found 2, breaking loop.")
        break
    print(f"Processing {num}")
else:
    print("All numbers processed.") # 此处 else 不会执行
```

4. **break, continue 和 pass 语句**这些语句用于在循环内部改变循环的正常流程。

- break 语句:

- 立即终止包含它的整个循环 (**for** 或 **while**)，程序将执行循环之后的下一条语句。
- 如果用于嵌套循环，**break** 只会跳出最内层的循环。

```
for i in range(10):
    if i == 5:
        break # 当 i 等于 5 时，跳出循环
```

```
print(i)
print("循环结束后的第一条语句")
# 输出:
# 0
# 1
# 2
# 3
# 4
# 循环结束后的第一条语句
```

- **continue 语句:**

– 立即结束当前迭代，跳过循环体中 `continue` 之后的剩余语句，直接开始下一次迭代。

```
for i in range(5):
    if i == 2:
        continue # 当 i 等于 2 时，跳过本次迭代的 print(i)
    print(i)
# 输出:
# 0
# 1
# 3
# 4
```

- **pass 语句:**

– `pass` 是一个空操作语句。它什么也不做。

– 当语法上需要一条语句，但程序不需要执行任何操作时，可以使用 `pass`。

– 常用作占位符，例如在定义一个空的函数或类时。

```
def my_empty_function():
    pass # 以后再来实现这个函数
```

```
if True:
    pass # 占位
else:
    print("False")
```

```
for i in range(3):
    if i == 1:
        pass # 逻辑上可能需要一个分支，但暂时不做任何事
```

```
else:
print(i)
# 输出:
# 0
# 2
```

九、函数 (Functions)

函数是一段组织好的、可重复使用的、用来实现单一或相关联功能的代码块。函数可以提高代码的模块化程度和复用性。

1. 定义函数

- 使用 `def` 关键字定义函数。
- `def` 后面跟着函数名和圆括号 `()`，圆括号内可以包含参数。
- 函数体的第一行可以选择性地使用文档字符串 (docstring) 来描述函数的功能。
- 函数体必须缩进。
- 使用 `return` 语句返回一个值。如果函数没有 `return` 语句，或者 `return` 后面没有表达式，它会自动返回 `None`。
- 基本语法:

```
def function_name(parameter1, parameter2, ...):
    """函数文档字符串 (可选)"""
    # 函数体代码
    # ...
    return [expression] # 可选的返回语句
```

- 示例:

```
# 定义一个简单的问候函数
def greet(name):
    """这个函数向指定的人问好。"""
    print(f"Hello, {name}!")

# 定义一个计算两数之和的函数
def add_numbers(x, y):
    """这个函数返回两个数字的和。"""
    sum_val = x + y
    return sum_val
```


2. 调用函数

- 定义函数后，可以通过函数名加上圆括号和必要的参数来调用它。

```
greet("Alice") # 调用 greet 函数，输出: Hello, Alice!

result = add_numbers(5, 3) # 调用 add_numbers 函数，并将返回值赋给 result
print(f"5 + 3 = {result}") # 输出: 5 + 3 = 8

# 如果函数没有返回值（或隐式返回 None）
output = greet("Bob")
print(output) # 输出: None（因为 greet 函数没有 return 语句）
```

3. 参数 (Parameters and Arguments)

- **参数 (Parameters):** 是在函数定义时列出的名称，它们是函数内部使用的变量。
- **实参 (Arguments):** 是在调用函数时传递给函数的值。
- **位置参数 (Positional Arguments):**
 - 调用函数时，实参按照它们在函数定义中参数的位置进行匹配。
 - 数量和顺序必须与定义时一致。

```
def describe_pet(animal_type, pet_name):
    print(f"I have a {animal_type}.")
    print(f"My {animal_type}'s name is {pet_name}.")
```

```
describe_pet("hamster", "Harry") # "hamster" 对应 animal_type, "Harry" 对应 pet_name
```

- **关键字参数 (Keyword Arguments):**
 - 调用函数时，以 参数名 = 值 的形式传递实参。
 - 使用关键字参数时，实参的顺序可以不与定义时一致。

```
describe_pet(pet_name="Lucy", animal_type="dog") # 顺序不重要
```

- **默认参数值 (Default Argument Values):**
 - 在函数定义时，可以为参数指定默认值。
 - 如果调用函数时没有为该参数提供实参，则使用默认值。
 - **重要:** 具有默认值的参数必须在没有默认值的参数之后。

```
def describe_pet_default(pet_name, animal_type="dog"): # animal_type 有默认值
    print(f"I have a {animal_type}.")
    print(f"My {animal_type}'s name is {pet_name}.")

describe_pet_default("Willie") # animal_type 使用默认值 "dog"
describe_pet_default("Whiskers", "cat") # animal_type 被指定为 "cat"
# describe_pet_default(animal_type="cat", "Mitty") # 错误! 位置参数不能在关键字参数之后
# (除非是像 def func(a, b="val", c) 这样是不允许的, 默认参数后面不能再有非默认参数)
# 应该这样: def func(a, c, b="val") 或者 def func(a, b="val", c="another_val")
```

- 可变参数 (Variable-length Arguments) (了解即可):

- `*args` (任意数量的位置参数): 将多余的位置参数收集为一个元组。
- `**kwargs` (任意数量的关键字参数): 将多余的关键字参数收集为一个字典。

```
def print_all_args(*args): # args 是一个元组
    print("位置参数:")
    for arg in args:
        print(arg)

print_all_args(1, "hello", True)

def print_all_kwargs(**kwargs): # kwargs 是一个字典
    print("关键字参数:")
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_all_kwargs(name="Alice", age=30, city="New York")
```

4. 返回值 (Return Statement)

- 函数使用 `return` 语句将结果返回给调用者。
- `return` 语句会立即终止函数的执行。
- 函数可以返回任何类型的数据, 包括列表、字典等复杂类型。
- 如果函数没有 `return` 语句, 或者 `return` 后面没有表达式, 它会隐式返回 `None`。
- 函数可以返回多个值, 这些值会以元组的形式返回。

```
def calculate_area(width, height):
    if width < 0 or height < 0:
```

```

return None # 或者可以引发一个错误
area = width * height
return area

area1 = calculate_area(5, 4) # area1 is 20
area2 = calculate_area(-2, 4) # area2 is None
print(area1, area2)

def get_name_and_age():
    name = "Bob"
    age = 25
    return name, age # 返回一个元组 ('Bob', 25)

person_name, person_age = get_name_and_age() # 元组解包
print(f"Name: {person_name}, Age: {person_age}")

```

5. 变量作用域 (Scope of Variables) 作用域是指变量在程序中可以访问的区域。

- 局部变量 (Local Variables):

- 在函数内部定义的变量，只在该函数内部有效。
- 当函数执行完毕后，局部变量通常会被销毁。
- 函数的参数也属于局部变量。

- 全局变量 (Global Variables):

- 在函数外部定义的变量，或者使用 `global` 关键字在函数内部声明的变量。
- 可以在程序的任何地方被访问（在定义之后）。

- `global` 关键字: 如果想在函数内部修改全局变量的值，必须使用 `global` 关键字声明该变量。

```

global_var = 100 # 这是一个全局变量

def my_function():
    local_var = 10 # 这是一个局部变量
    print(f"Inside function: local_var = {local_var}")
    print(f"Inside function: global_var = {global_var}") # 可以访问全局变量

my_function()
print(f"Outside function: global_var = {global_var}")
# print(f"Outside function: local_var = {local_var}") # 这会引发 NameError, 因为 local_var 在函数外部未定义

# 修改全局变量
count = 0

```

```
def increment_count():
    global count # 声明 count 是全局变量
    count += 1
    print(f"Count inside function: {count}")

increment_count() # Count inside function: 1
increment_count() # Count inside function: 2
print(f"Count outside function: {count}") # Count outside function: 2
```

- **LEGB 规则**: Python 查找变量名的顺序是: Local (局部) → Enclosing function locals (嵌套函数作用域) → Global (全局) → Built-in (内置作用域)。

6. 一些重要的内置函数 (**Built-in Functions**) Python 提供了许多可以直接使用的内置函数。以下是一些常用的:

- `print()`: 打印输出到控制台。
- `input()`: 从用户获取输入。
- `len(s)`: 返回对象 `s` (如字符串、列表、元组、字典) 的长度或元素个数。
- `type(object)`: 返回对象的类型。
- `int(x)`, `float(x)`, `str(x)`, `bool(x)`: 进行类型转换。
- `list(iterable)`, `tuple(iterable)`, `dict(...)`: 创建列表、元组、字典。
- `sum(iterable[, start])`: 返回可迭代对象中所有元素的和。
- `max(iterable)` 或 `max(arg1, arg2, *args)`: 返回最大值。
- `min(iterable)` 或 `min(arg1, arg2, *args)`: 返回最小值。
- `range(start, stop[, step])`: 生成一个整数序列。
- `abs(x)`: 返回数字 `x` 的绝对值。
- `round(number[, ndigits])`: 对数字进行四舍五入到指定的小数位数 `ndigits`。
- `sorted(iterable, key=None, reverse=False)`: 返回一个新的已排序的列表 (不修改原对象)。
- `isinstance(object, classinfo)`: 判断一个对象是否是指定类或类型的实例。

```
print(len("hello"))    # 5
print(type(123))       # <class 'int'>
numbers = [1, 5, 2, 8]
print(sum(numbers))    # 16
print(max(numbers))    # 8
```

```
print(min(numbers))    # 1
print(abs(-10))        # 10
print(round(3.14159, 2)) # 3.14
print(sorted("banana")) # ['a', 'a', 'a', 'b', 'n', 'n']
```

— 祝考试顺利 —